클라우드 엔지니어링 6기

멋쟁이사자처럼

26.03.31 (화) | 정범석님

자바 제어문과 반복문 기초

👉 오늘의 강의 학습 요약

이번 강의는 어제 학습한 데이터/변수/형변환의 핵심을 복습한 뒤, 제어문을 조건문(분기문)과 반복문으로 크게 나누어 구조적으로 정리했습니다. 특히 기본형/참조형 변수의 메모리 관점 차이, 변수 종류(로컬/인스턴스/클래스)와 초기화 규칙, 그리고 자바의 블록 스코프가 제어문 블록과 직접적으로 연결된다는 점을 강조했습니다.

이어서 조건문은 단일 `if`, `if-else`, 다중 `if-else if-else`, `switch`로 구분하여 문법과 실행 흐름을 실습 중심으로 확인했고, `switch`의 fall-through와 `break`의 역할, 사용 가능한 타입 제한, 그리고 JDK 14+의 `switch expression`까지 다뤘습니다. 마지막으로 반복문은 반복을 위해 필요한 3요소(초기값, 조건식, 증감식)를 먼저 이해한 뒤, 이를 `for` 문법에 배치하는 방식과 동작 순서를 설명하고, 변형/중첩/오류(미실행, 무한 루프) 사례 및 간단 과제를 제시했습니다.

1. 변수, 메모리, 스코프 핵심 복습

> **기본형/참조형 변수의 저장 방식 차이, 변수 선언, 초기화 규칙, 로컬, 인스턴스, 클래스 변수의 생명주기 및 자동 초기화, 그리고 자바의 블록 스코프를 제어문과 함께 적용하는 관점이 핵심입니다.**

데이터는 프로그램에서 가장 핵심이며, 자바 데이터는 기본형 8개와 참조형으로 구분됩니다. 변수는 데이터를 저장해 재사용하기 위한 "메모리 영역"이며, 기본형 변수는 실제 값이 저장되고 참조형 변수는 실제 값이 아니라 주소값이 저장된다는 점이 본질적인 차이입니다.

구분	변수에 저장되는 것	메모리 관점 핵심
기본형 변수	실제 값(value)	타입별 크기(예: `int` 4B, `double` 8B)에 맞게 값이 직접 저장됩니다.
참조형 변수	주소값(reference)	참조 변수 자체는 고정 크기(강의 기준 4B)이며, 주소를 따라가야 실제 데이터에 접근됩니다.

변수를 만드는 과정은 "선언(영역 확보)"과 "초기화(처음 값 대입)"로 정리됩니다. 선언과 초기화는 분리할 수도 있고 한 줄로 결합할 수도 있으며, 변수는 필요 시 값이 변경될 수 있다는 특성 때문에 변수라고 부릅니다.

변수는 선언 위치에 따라 로컬 변수, 인스턴스 변수, 클래스(스태틱) 변수로도 분류됩니다. 이 분류는 코드에서 위치로 구분할 수 있어야 하며, 생성/소멸 시점과 초기화 규칙이 달라 실제 버그와 직결됩니다.

변수 종류	선언 위치	생성/소멸 시점	초기화 규칙	저장 영역(강의 관점)
로컬 변수	메소드 내부	메소드 호출 시 생성, 메소드 종료 시 소멸	반드시 명시적 초기화가 필요합니다.	스택(Stack)
인스턴스 변수	메소드 밖(클래스 멤버), `static` 없음	`new` 로 객체 생성 시 생성	자동 초기화됩니다.	힙(Heap)
클래스 변수	메소드 밖(클래스 멤버), `static` 있음	클래스 로딩/사용 시점에 준비	자동 초기화됩니다.	(개념 정리는 추후 확장)

로컬 변수는 초기화하지 않으면 컴파일러가 오류를 잡아주며, 인스턴스/클래스 변수는 자동 초기화가 적용됩니다. 자동 초기화의 대표 예시는 정수 `0`, 실수 `0.0`, 논리 `false`, 참조형 `null` 이며 `null` 은 "가리키는 주소가 없음"을 의미합니다.

자바는 블록 스코프를 따르므로 중괄호 `{}` 블록 안에서 선언한 변수는 블록 밖에서 사용할 수 없습니다. 따라서 `if`, `switch`, 반복문 같은 블록 구조에서 "블록 밖에서도 써야 하는 값"이라면 변수를 바깥에서 선언하고 블록 안에서는 할당만 수행하도록 설계해야 합니다.

```
int num = 0;          // 블록 밖에서 선언(로컬 변수이므로 초기화 필요)
if (true) {
  num = 10;           // 블록 안에서는 값만 대입
}
System.out.println(num); // 블록 밖에서도 사용 가능
```

상수는 변수와 달리 값을 변경할 수 없으며, 자바에서는 `final` 키워드로 상수화합니다. 관례적으로 상수명은 "모든 글자를 대문자"로 작성해 변수와 구분하며, 이런 네이밍 관례는 협업 가독성과 오해 방지에 직접적 영향을 줍니다.

형변환은 자동 형변환과 강제 형변환으로 나뉘며, 기본형뿐 아니라 참조형도 조건을 만족하면 형변환이 가능합니다. 특히 참조형 형변환은 "비슷한 성질의 클래스"에서 가능하며, 이는 이후 상속(부모/자식 관계) 개념과 연결됩니다.

구분	핵심 규칙(강의에서 언급된 자동 형변환 포인트)
수치 타입 확장	작은 타입에서 큰 타입으로 자동 변환될 수 있습니다.
`char` 변환	`char` 는 `int` 로 자동 변환될 수 있습니다.
정수 연산 승격	`int` 보다 작은 정수 타입 간 연산 결과는 `int` 로 나올 수 있습니다.
문자열 결합	문자열과 비문자열을 `+` 로 연결하면 결과는 문자열이 됩니다.

강제 형변환은 `(타입)` 형태의 형변환 연산자를 사용하며, 변환 가능 여부는 데이터 성질(유사성)에 의해 좌우됩니다.

핵심 키워드 #기본형 #참조형 #블록스코프 #null #형변환

2. 조건문(분기문) 정리와 실습

> **제어문은 조건식에 따라 실행 흐름이 갈라지거나 반복되도록 만드는 문장이고, 분기문은 단일 if, if-else, 다중 if, switch로 구분되며 블록과 스코프 규칙을 함께 적용해야 합니다.**

제어문은 순차 실행만으로는 표현하기 어려운 "조건에 따른 실행 경로 변화" 또는 "반복 실행"을 구현합니다. 분기문은 조건에 따라 실행되는 문장이 달라지는 구조이며, 반복문은 조건에 따라 동일 문장이 여러 번 실행될 수 있는 구조입니다.

분기문은 크게 단일 `if`, `if-else`, 다중 `if-else if-else`, `switch`로 정리됩니다. 단일 `if`는 특정 문장(또는 문장 그룹)을 조건에 따라 실행할지 말지를 결정하며, 조건식이 `true`일 때만 실행됩니다.

단일 `if`에서 중괄호는 문법상 생략 가능하지만, 가독성과 확장(문장 그룹화) 관점에서 "문장이 1개여도 중괄호 사용을 권장"합니다. 중괄호를 생략하면 `if` 바로 다음 문장 1개만 조건의 영향을 받습니다.

```
if (condition) {
  // 조건이 true일 때만 실행되는 문장(1개 이상 가능)
}
```

`if-else`는 조건에 따라 실행되는 문장이 "서로 다른 두 갈래"로 나뉘는 경우에 사용합니다. 관례적으로 `if`의 중괄호 블록을 `if 블록`, `else`의 중괄호 블록을 `else 블록`이라고 부르며, 조건식 결과에 따라 둘 중 하나만 실행됩니다.

다중 `if`(= `if-else if-else`)는 조건이 여러 개인 상황을 순차적으로 검사하는 구조입니다. 특정 분기(`else if` 블록)가 실행되려면 그 이전의 조건들이 모두 `false`여야 하며, 마지막 `else`는 "어떤 조건도 만족하지 않을 때"의 기본 처리로 사용됩니다.

강의 실습 예시는 점수를 입력받아 학점을 출력하는 형태였습니다. 범위 비교는 비교 연산자 및 논리 연산자를 조합하며, 상한/하한을 모두 비교하는 방식과 상한을 전제하고 하한만 비교하는 방식이 모두 가능하다고 정리되었습니다.

분기	대표 조건 예시	처리 예시
A	`score >= 90 && score <= 100` 또는 `score >= 90` (상한 전제)	`"A학점"` 출력입니다.
B	`score >= 80 && score < 90`	`"B학점"` 출력입니다.
C	`score >= 70 && score < 80`	`"C학점"` 출력입니다.
F	위 조건을 모두 만족하지 않는 경우	`"F학점"` 출력입니다.

`switch` 문은 조건이 여러 개인 경우에 사용하지만, 내부 비교가 "동등 비교(==) 기반"이라는 제약이 핵심 차이입니다. 즉 `switch` 는 크다/작다 같은 부등 비교를 직접 표현할 수 없으므로, 범위 조건이 필요하면 다중 `if` 가 더 적합합니다.

`switch` 의 기본 문법은 `switch(변수){ case 값: ... default: ... }` 형태이며, `case` 는 여러 개를 둘 수 있고 `default` 는 어떤 값에도 일치하지 않을 때 실행되는 기본 분기입니다. 또한 `break` 가 없으면 일치한 `case` 부터 아래로 "주르륵(fall-through)" 실행되는 것이 기본 동작이므로, 일반적인 `switch` 사용에서는 각 `case` 에 `break` 를 두어 해당 분기만 실행되도록 합니다.

요소	의미	주의점
`case`	비교 값에 일치할 때 실행할 분기입니다.	`break` 가 없으면 아래 분기까지 연속 실행됩니다.
`break`	`switch` 블록을 즉시 빠져나옵니다.	필수는 아니며 의도적으로 fall-through를 만들 수도 있습니다.
`default`	어떤 `case` 에도 매칭되지 않을 때 실행됩니다.	보통 마지막에 배치해 예외 흐름을 수용합니다.

`switch` 에서 사용할 수 있는 변수 타입은 제한이 있으며, 강의에서는 6개 타입으로 정리되었습니다. 실수형 및 `long` 은 사용할 수 없고, `byte` , `short` , `int` , `char` , `String` , 그리고(추후 학습할) 특정 타입이 가능하다고 정리되었습니다.

사용 가능(강의 기준)	사용 불가(강의에서 확인)
`byte` , `short` , `int` , `char` , `String` , (추후 학습 타입)	`long` , 모든 실수형(`float` , `double`)

추가로, JDK 14+에서 도입된 `switch expression` 문법을 소개했습니다. 이 문법은 `case 값 -> 반환값` 형태로 값을 반환하며, 변수에 대입할 수 있고, fall-through 없이 "하나가 매칭되면 그 자리에서 종료되는" 특징을 가집니다.

```
int result = switch (n) {
  case 10 -> 100;
  case 20 -> 200;
  default -> 0;
};
```

핵심 키워드 `#단일if` `#if-else` `#다중if` `#switch` `#switch expression`

3. 반복문: for문 구조와 동작 원리

> **반복을 위해서는 초기값, 조건식, 증감식의 3요소가 필수이며, for문은 이 3요소를 괄호 안에 배치하고 블록의 문장을 반복 실행하며, 조건 오류는 미실행 또는 무한 루프의 원인이 됩니다.**

반복문은 조건에 따라 같은 문장을 여러 번 실행할 수 있게 하며, 자바에서는 `for`, `while`, `do-while`이 존재합니다. 강의에서는 반복을 설계하기 위한 핵심 정보가 3가지로 고정된다는 점을 먼저 정의했습니다.

반복 3요소	역할	누락/오류 시 결과
초기값	반복을 시작하는 기준 값입니다.	시작 자체가 의도와 달라지거나 반복 횟수가 어긋납니다.
조건식	반복을 계속할지 멈출지 결정합니다.	처음부터 `false`면 0회 실행이 됩니다.
증감식	조건식이 언젠가 `false`가 되도록 값을 변화시킵니다.	조건이 영원히 `true`면 무한 루프가 됩니다.

`for` 문은 이 3요소를 괄호 안에 `초기값; 조건식; 증감식` 순서로 배치하고, 중괄호 블록 내부 문장을 반복합니다. 예시로 "Hello 5번 출력"은 다음과 같이 구성되며, 조건을 `< 6` 또는 `<= 5`처럼 다양하게 표현할 수 있습니다.

```
for (int n = 1; n < 6; n++) {
  System.out.println("Hello");
}
```

실행 순서는 "초기값 1회 실행 → 조건식 검사 → 블록 실행 → 증감식 실행 → 조건식 재검사"로 반복되며, 조건식이 `false`가 되는 순간 반복문을 빠져나옵니다. 또한 초기값은 반복 중 다시 실행되지 않고 최초 1번만 실행된다는 점이 중요합니다.

`for` 문은 표준 형태 외에도 다양한 변형이 가능합니다. 시작값을 0이나 100으로 바꾸거나, 감소(`--`)로 내려가게 만들거나, 2씩/3씩 증가시키는 등 증감 폭을 조정할 수 있으며, 초기값/증감식에 변수를 2개 이상 두고 조건식에 논리 연산자를 결합하는 형태도 가능합니다.

변형 포인트	예시 방향	의미
시작값 변경	0부터 시작, 100부터 시작	반복 인덱스의 기준을 업무 의미에 맞게 조정합니다.
감소 반복	9,8,7,6...	역순 처리나 뒤에서 앞으로 탐색에 활용됩니다.
증감 폭 변경	2씩 증가(0,2,4,6,8)	짝수만 순회, 샘플링, 간격 처리 등에 활용됩니다.
다중 변수/조건	변수 2개, 조건 2개	복합 상태를 함께 갱신하며 반복 제어가 가능합니다.

반복문 중첩도 가능하며, 반복문 안에 또 다른 반복문을 넣는 구조로 "바깥 반복 5회 × 안쪽 반복 5회"처럼 전체 반복 횟수가 곱으로 늘어날 수 있습니다. 또한 조건문과 반복문은 서로 중첩될 수 있어 `if` 안에 `for`, `for` 안에 `if` 같은 조합이 모두 가능합니다.

반복이 아예 실행되지 않는 사례는 "초기값과 조건식 조합이 처음부터 `false` 인 경우"이며, 무한 루프는 "증감식 누락 또는 조건이 `false` 로 바뀌지 않는 경우"에 발생합니다. 이때는 실행을 중단(예: 실행 콘솔에서 중지)해야 하며, 반복 3요소를 다시 점검하는 것이 핵심 디버깅 관점입니다.

마지막으로 실습 과제로 `1\~20` 범위를 반복하며 (1) 짝수만 출력, (2) 3의 배수만 출력, (3) 총합 계산 같은 패턴을 제시했습니다. 이 과제는 `for` 의 조건식/증감식과 `if` 조건 결합을 연습하기 위한 전형적인 문제 구성이 됩니다.

핵심 키워드 #반복문 #for문 #초기값 #조건식 #증감식